LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack and Queue

Oleh:

Achmad Sajidi Aziz NIM. 2510817210025

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Achmad Sajidi Aziz
NIM : 2510817210025

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
Soal 1: Implementasi Stack 6
A. Source Code 7
B. Pembahasan 8
Soal 2: Si Palui dan Mesin Hitung Aneh 9
A. Source Code 9
B. Output Program 11
C. Pembahasan 11
Soal 3: Implementasi Queue 12
A. Source Code 12
B. Pembahasan 14
Soal 4: Si Palui dan Deretan Angka 16
A. Source Code 16
B. Output Program 18
C. Pembahasan 18
TAUTAN GIT 19

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 11
Gambar 2 Screenshot Hasil Jawaban Soal 4 18

DAFTAR TABEL

Tabel 1 Source Code stack.cpp..... 7
Tabel 2 Source Code prob1.cpp 9
Table 3 Source Code queue.cpp 12
Table 4 Source Code prob2.cpp 16

Soal 1: Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
    return s->top >= s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
          throw std::overflow_error("Stack penuh, tidak bisa push");
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
         throw std::underflow_error("Stack kosong, tidak bisa pop");
    }
    s->top--;
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
         throw std::underflow_error("Stack kosong, tidak ada elemen");
    }
    return *(s->top);
}
```

B. Pembahasan

Pertama, dalam fungsi `init(Stack* s)` menempatkan top sebelum elemen pertama yang berarti stack masih kosong, jika `top < data` berarti tidak ada elemen yang valid. Fungsi ini hanya menggunakan satu penugasan, sehingga kompleksitas waktu nya : O(1). Fungsi ini juga hanya menggunakan variabel lokal tanpa alokasi memori tambahan, sehingga kompleksitas ruang nya juga O(1).

Kedua, dalam fungsi `isEmpty` terdapat `return s->top < s->data` yang mana jika ada elemen `top ≥ data` maka ini bernilai FALSE dan `top < data` bernilai TRUE jika tidak ada elemen. Definisi ini konstan dengan `init()`. Kompleksitas waktu dan ruangnya sama sama O(1), hanya menggunakan satu operasi perbandingan pointer dan membacanya.

Ketiga, dalam fungsi `isFull` terdapat `return s->top >= s->data + MAX - 1` yang mengecek apakah poissi top sudah mencapai batas akhir array. Jika sudah mencapai atau melewati batas tersebut maka stack dianggap penuh. Operasi ini bersifat statis karena hanya membandingkan alamat pointer dengan batas maksimum stack. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya melakukan satu perbandingan dan operasi aritmatika pointer.

Keempat, dalam fungsi `push` dilakukan pengecekan apakah stack penuh melalui `isFull`, kemudian jika tidak penuh maka `s->top` dinaikkan dan nilai dimasukkan ke alamat tersebut. Operasi ini mengikuti prinsip LIFO dimana elemen baru selalu ditempatkan di posisi paling atas. Karena semua langkah hanya berupa pengecekan, increment pointer, dan assignment nilai tanpa loop, maka kompleksitas waktu dan ruangnya sama-sama O(1).

Kelima, dalam fungsi pop terdapat pengecekan apakah stack kosong melalui `isEmpty`, kemudian jika tidak kosong maka pointer top diturunkan satu posisi. Elemen tidak benar-benar dihapus secara fisik, tetapi cukup dengan menggeser batas stack. Karena hanya melakukan satu pengecekan dan satu operasi decrement pointer tanpa iterasi atau alokasi tambahan, maka kompleksitas waktu dan ruangnya sama-sama O(1).

Keenam, dalam fungsi peek terdapat pengecekan `isEmpty`, kemudian jika tidak kosong maka nilai pada posisi top dikembalikan tanpa mengubah struktur stack. Fungsi ini hanya membaca elemen teratas yang sudah diketahui posisinya melalui pointer top, sehingga tidak perlu pencarian. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya melakukan pengecekan dan akses langsung ke elemen tanpa alokasi atau iterasi tambahan.

Soal 2: Si Palui dan Mesin Hitung Aneh

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.
Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2 Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include <stdexcept>
#include "stack.h"

int main() {
    int n;
    std::cin >> n;

    Stack s;
    init(&s);

    try {
```

```
        for (int i = 0; i < n; i++) {
            std::string A;
            std::cin >> A;

            if (A == "+" || A == "-" || A == "*" || A == "/") {
                int b = peek(&s); pop(&s);
                int a = peek(&s); pop(&s);

                int hasil;

                if (A == "+") hasil = a + b;
                else if (A == "-") hasil = a - b;
                else if (A == "*") hasil = a * b;
                else hasil = a / b;

                push(&s, hasil);
            }
            else {
                int angka = std::stoi(A);
                push(&s, angka);
            }
        }
        std::cout << peek(&s) << std::endl;

    } catch (const std::overflow_error& e) {
        std::cout << "error: " << e.what() << std::endl;
    } catch (const std::underflow_error& e) {
        std::cout << "error: " << e.what() << std::endl;
    } catch (const std::exception& e) {
        std::cout << "error: " << e.what() << std::endl;
    }

    return 0;
}
```

B. Output Program

Gambar 1 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [3], `#include <stdexcept>` dipakai untuk menangani error seperti stack penuh atau kosong.

Pada baris [10-11], membuat stack dan inisilisasi yang nantinya semua proses akan bergantung pada stack.

Pada baris [13], `try` digunakan agar program tetap aman jika terjadi error.

Pada baris [14-37], program perulangan sebanyak n kali untuk memproses input. Setiap iterasi, string A dari input karena format input berupa angka maupun operator(string fleksibel). Program mencek A apakah operator, jika operator maka program mengambil dua niai teratas dari stack menggunakan peek dan pop, nilai pertama=b; nilai kedua=a. Lalu program menghitung sesuai operator dan hasilnya disimpan di variabel hasil. Hasil tadi dimasukkan lagi ke stack menggunkan push supaya hasil bisa digunakan kembali jika ada operasi berikutnya. Jika A bukan operator, berarti angka. Konversi string menjadi int menggunakan `std::stoi` dan masuk ke stack. Proses berulang sampai semua input sudah diproses, lalu menampilkan hasil akhir berupa bilangan hasil perhiutngan.

Pada baris [38-44], berisi `catch` untuk menangi error.

Soal 3: Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 3 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    int frontIndex = q->front - q->data;
    int rearIndex  = q->rear  - q->data;

    return (rearIndex + 1) % MAX == frontIndex;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::runtime_error("Queue full");
    }

    if (isEmpty(q)) {
        q->front = q->rear = &q->data[0];
        *q->rear = value;
    } else {
        int rearIndex = (q->rear - q->data + 1) % MAX;
        q->rear = &q->data[rearIndex];
        *q->rear = value;
```

```
    }
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue empty");
    }

    if (q->front == q->rear) {
        init(q);
    } else {
        int frontIndex = (q->front - q->data + 1) % MAX;
        q->front = &q->data[frontIndex];
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue empty");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue empty");
    }
    return *(q->rear);
}
```

B. Pembahasan

Pertama, dalam fungsi `init` terdapat inisialisasi `q->front = nullptr dan q->rear = nullptr` yang menandakan queue berada dalam kondisi kosong sejak awal. Dengan kedua pointer bernilai nullptr, struktur queue tidak memiliki elemen aktif. Proses ini hanya berupa dua operasi penugasan tanpa ketergantungan pada ukuran data atau elemen yang ada. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya melakukan assignment tetap tanpa iterasi maupun alokasi tambahan.

Kedua, dalam fungsi `isEmpty` terdapat return `q->front == nullptr` yang mengecek apakah queue kosong berdasarkan kondisi pointer front. Jika front bernilai nullptr, maka queue dianggap kosong, sedangkan jika tidak maka queue memiliki elemen. Definisi ini konsisten dengan inisialisasi pada fungsi init. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya menggunakan satu operasi perbandingan pointer tanpa proses tambahan.

Ketiga, dalam fungsi `isFull` dilakukan pengecekan kondisi circular queue dengan menghitung indeks front dan rear, lalu membandingkan `(rearIndex + 1) % MAX == frontIndex`. Jika kondisi tersebut terpenuhi, maka queue dianggap penuh. Operasi ini bersifat konstan karena hanya melibatkan beberapa operasi aritmatika dan satu perbandingan, tanpa iterasi terhadap elemen queue. Kompleksitas waktu dan ruangnya sama-sama O(1) karena jumlah operasi tidak bergantung pada banyaknya elemen.

Keempat, dalam fungsi `enqueue` dilakukan pengecekan apakah queue penuh, kemudian jika kosong maka front dan rear diinisialisasi ke awal array, dan jika tidak kosong maka rear digeser secara circular menggunakan modulo. Setelah itu nilai dimasukkan ke posisi rear. Operasi ini mengikuti prinsip FIFO di mana elemen baru selalu ditambahkan di belakang. Karena semua langkah hanya berupa pengecekan kondisi, perhitungan indeks, pointer assignment, dan penulisan nilai tanpa looping, maka kompleksitas waktu dan ruangnya sama-sama O(1).

Kelima, dalam fungsi `dequeue` dilakukan pengecekan apakah queue kosong, kemudian jika hanya terdapat satu elemen maka queue di-reset menggunakan init, sedangkan jika lebih dari satu elemen maka front digeser maju secara circular menggunakan operasi modulo. Elemen tidak dihapus secara fisik, hanya batas akses yang berubah. Karena operasi hanya mencakup pengecekan kondisi dan satu perubahan pointer tanpa iterasi, maka kompleksitas waktu dan ruangnya sama-sama O(1).

Keenam, dalam fungsi `front` terdapat pengecekan `isEmpty`, kemudian jika tidak kosong maka nilai pada posisi `front` dikembalikan tanpa mengubah struktur queue. Fungsi ini hanya mengakses elemen pertama yang sudah diketahui melalui pointer front, sehingga tidak memerlukan pencarian. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya melakukan pengecekan dan akses langsung ke elemen.

Ketujuh, dalam fungsi `back` terdapat pengecekan `isEmpty`, kemudian jika tidak kosong maka nilai pada posisi rear dikembalikan tanpa perubahan struktur queue. Karena pointer rear selalu menunjuk elemen terakhir yang valid, maka tidak diperlukan traversal untuk mendapatkan nilai tersebut. Kompleksitas waktu dan ruangnya sama-sama O(1) karena hanya melakukan satu pengecekan dan satu akses langsung ke memori tanpa alokasi tambahan.

Soal 4: Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.
Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.
Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.
Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 4 Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

int main() {
    int n, k;
    std::cin >> n >> k;

    if (k <= 0 || k > n) {
        std::cout << "error" << std::endl;
        return 0;
    }

    Queue q;
    init(&q);

    int sum = 0;
    int x;

    try {
        int i = 0;
        while (i < k) {
            std::cin >> x;
```

```
            enqueue(&q, x);
            sum += x;
            i++;
        }

        std::cout << sum;

        while (i < n) {
            std::cin >> x;

            int depan = front(&q);
            dequeue(&q);
            sum -= depan;

            enqueue(&q, x);
            sum += x;

            std::cout << " " << sum;
            i++;
        }
        std::cout << std::endl;

    } catch (const char* msg) {
        std::cout << "error: " << msg << std::endl;
    }

    return 0;
}
```

B. Output Program

```
5 3
1 2 3 4 5
6 9 12
```

```
5 5
1 2 3 4 5
15
```

```
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 2 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [8-11], program validasi jika k <= 0 atau k > n, langsung cetak "error" dan berhenti. Karena tidak masuk akal jika nilai k segitu.

Pada baris [13-14], membuat queue dan inisialisasi yang nantinya akan digunakan untuk menyimpan elemen-elemen.

Pada baris [16-17], sum untuk manyimpan jumlah elemen dan x untuk menampung input.

Pada baris [20-41], queue diisi dengan k elemen awal menggunakan perulangan, setiap elemen yang dibaca dimasukkan ke enqueue dan ditambah ke sum untuk membentuk jendela pertama berisi k elemen lalu sum pertama ditampilkan untuk hasil jendela awal. Selama masih ada data hingga n, program menggeser jendela. Setiap elemen baru, elemen paling depan queue diambil menggunakan front lalu dihapus dengan dequeue, nilainya lalu dikurangkan dari sum dan memasukkan elemen baru ke enqueue lalu ditambah ke sum. Jadi, proram mengurangi elemen lama dan menambahkan elemen baru. Setelah selesai, sum dicetak berupa deretan jumlah dari setiap kelompok k elemen yang berurutan.

Pada baris [45-47], `catch` akan menangkap dan menampilkan pesan error jika ada error dalam operasi queue.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-asajidiaziz0-alt.git